

I. Collaboration Details: Description of Contribution of Each Team Member.

Selina Wu: PyLucene setup, debugging, implemented flask, search engine, designed input/output html files, report, video

Nada Salib: PyLucene setup, debugging, implemented flask, search engine, design input/output html files, video

Lindsay Adams: PyLucene, handled duplicates, debugging, implemented flask, extra credit, report, video

Ananya Sood: PyLucene, implemented order for time/relevance, debugging, implemented flask, extra credit, report, video

Rhea Niraj: Trying to debug code issues, video

II. Overview of system, including (but not limited to):

1. Architecture

Our project is a web-based reddit search interface using PyLucene, where we retrieve and rank Reddit posts based on relevance, recency, and popularity. Our architecture has three main parts, data preprocessing and indexing, search and ranking logic, and the web interface. Our reddit posts are taken from the JSON files from part 1, processed, and indexed using Lucene. Each post is indexed, as it has the title, body, comments, upvotes, timestamp, and any other data making it efficient for querying.

Our backend logic is mainly implemented in the pylucene_reddit.py file, which essentially handles indexing and retrieval. In our file, we have a create_index function that essentially builds the Lucene index with field types for each attribute in Reddit. Essentially, when a user submits a search query, the retrieve function uses the IndexSearcher in Lucene and BM25Similarity to get the relevant documents. The resulting documents are ranked by both the Lucene relevance scores and recency, which is the timestamp, and upvotes. We also have a custom ranking function that will essentially support multiple ranking modes of “relevance”, “time”, and “votes” or a weighted combination of all, to ensure that there is flexibility of the prioritization of the search results.

The next part of our project is the web interface that is implemented using Flask. Our frontend has two templates, the input.html for the search engine form part of the project and the output.html that displays the results. The flask portion of the project essentially handles the user input from the form, extracts the query from it and the ranking parameters, and puts them into the retrieval function. Then, the results are taken into the output page, with the metadata of the scores and some of the reddit post’s body, making the interface clean, information and easy to navigate.

Therefore, this architecture creates a scalable and easy to navigate search system that uses Lucene for the enhancement of information retrieval. Though the use of combining relevance, recency, and upvotes for ranking, our implementation allows for a nuanced reddit content exploration. Our design also allows us to have any enhancements to our project if needed.

2. Index Structures

Our reddit search system uses an inverted index structure that is built using PyLucene. Each reddit post is parsed from a JSON line format and then indexed with a set of fields that show the content and metadata of the post. The index stores token and document references so that the search can be efficient across all entries. Our system uses the field analyzer wrapper so that the search precision and field specific weighting is maximized so that there is a tailored analysis for fields like comments and the author names.

A key part of our design for the indexing was making sure that the field types and indexing has options. This allows the “Title” and “Body” fields to be stored and tokenized so that they can be displayed and used in the relevance scoring. Additionally, the comments are indexed but not stored, while the “Upvotes” fields handle numeric data for scoring. This basically allows each piece of information to be used for retrieval, relevance, ranking, and the inference.

Additionally, we have a `create_index()` function that converts the raw reddit data into Lucene objects. Each document has fields with different configurations. This allows whether the field is tokenized, indexed or stored. Then the BM25Similarity is applied so that the relevance ranking is optimized based on term frequency and inverse document frequency. Basically, the similarity function allows for the inverted index to score the documents efficiently from the user queries.

After indexing, the `retrieve()` function basically executes the search process as it parses the user query, making sure that there is a search across the different fields, like “title,” “body,” and “comments.” Then the inverted index is accessed so that it can return the ranked list of candidate documents using the BM25 scores. Essentially, this allows for the enhancement of ranking as the new metadata is incorporated (recency (post age), and upvotes) into the final scoring function. Then the user has the ability to select the ranking mode, like relevance, time, votes, or a weighted combination of those three to sort the results dynamically at query time.

Essentially, the indexing design that we put into place allows for a flexible and sizable search platform. The inverted index architecture allows for fast retrieval, as the field types and the integration of the metadata allows for complex ranking to be put into place. Through this structure, basic keyword matching, relevance tuning is supported, and popularity filtering is supported allowing for meaningful information to be drawn from the surfaced Reddit content

3. Search Algorithm & Scoring Methods

The search algorithm uses Lucene’s BM25Similarity for ranking documents based on term frequency and inverse document frequency across the different fields. Essentially, when a user submits a query through the web interface, the search engine parses it so that it can look across the different fields simultaneously. This essentially allows flexibility when looking at relevance as some Reddit content can have integral information written in its title or body. After the initial search is executed, the system gets the top 30 candidate documents and their corresponding scores, showing only the top 10

documents in our inference. This ensures that our ranking quality is efficient and accurate through our custom ranking function.

It is worth noting that to implement the extra credit portion of this project, our scoring method scores the posts by relevance, recency, votes, or a combination of them (weight for each of the three factors). This allows our ranking to be even more accurate based on the different categories the user wants to see the documents by, allowing for a better search engine overall.

When ranking by relevance, we use Lucene's built-in BM25 scoring method, which chooses posts that closely match the user's search terms in the title, body, or comments. For recency (time), newer posts are given higher scores, and older posts gradually receive lower scores to ensure that fresh content is prioritized. If the user chooses to rank by votes, posts with more upvotes are ranked higher, but we use a scaled approach so that one very popular post doesn't overshadow everything else. Finally, our combined uses all three options (relevance, recency, and upvotes) using a weighted formula.

As mentioned above, the algorithm supports multiple ranking methods, as the user can sort the search engine results based on relevance, time, or upvotes or choose a weighted combination of the three. If the combined option is chosen, then the system normalizes the weights from the user and calculates the final score using the weighted sum of the three different components $((w_r \times \text{BM25 Score}) + (w_t \times \text{Recency Score}) + (w_v \times \text{Vote Score}))$, where the weights w_r , w_t , and w_v are user-defined (or default) values that are normalized to sum to 1). This essentially allows the user to control how much of an effect different factors have on their search. It allows for user-centered flexibility in this search engine.

To ensure clean results, our algorithm filters out duplicates based on post titles and sorts the rest of the results in descending order of final score, showing the results to the user. Each result has the snippet of the post body, basic metadata, and the full reddit post (if clicked). Essentially, our search algorithm creates a balance between performance, relevance, and user usability so that the search engine is efficient and properly ranks the reddit content.

III. Limitations of the System.

A limitation of the system is that it only uses keywords for the ranking function. Even though the BM25 is good for textual relevance, it does not show the relationship and context between terms, like synonyms and phrasing. This results in users having results that are relevant technically due to the keyword match, but could be not as conceptually meaningful. Even the extra credit portion, where we combine the ranking function with different categories and weights will not always work for different queries. The implementation of a more advanced learning ranking function would allow for results to be improved.

Another limitation is how we currently handle snippets and duplicate results. Right now, the snippet is just a cut-off of the post body, so it doesn't always show the most useful or relevant part of the post to the user. It would be better to have the snippet highlighted or extract the part of the body that actually matches the query. This would make the results more informative and help

users decide which post to click on. Essentially, our snippet generation is simple, and while it works, it can be improved to make the search results cleaner and more helpful.

IV. Instructions on how to deploy the system. Ideally, you should include an `indexer.bat` (Windows) or `indexer.sh` (Unix/Linux) executable file that takes as input all necessary parameters Example: `[user@server] ./indexer.sh <output-dir>`

install python3

change directory into CS172-Project: `cd CS172-Project`

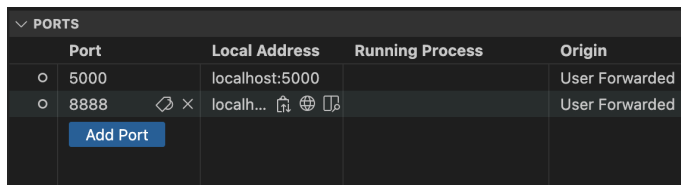
run the pylucene file (to test retrieval or build the index): `python3 pylucene_reddit.py`

this will create `__pycache__` folder:

change directory into flask_lucene_demo: `cd flask_lucene_demo`

run the flask file: `python3 flask_demo.py`

This will launch the web application locally on port 8888.



Port	Local Address	Running Process	Origin
5000	localhost:5000		User Forwarded
8888	localhost:5000		User Forwarded

Click on the circle Web Globe Icon to open the web-based application

You should now see the website opened in another tab!

V. A web-application (e.g. Web Archive) that can be deployed to a web server (Tomcat)

Our web application is built using Flask as the web framework. We developed and tested the application locally using VSCode, so that the functional application is ready to be deployed. The application allows the users to input a search query through our easy to navigate interface and returns a ranked list of reddit posts based on that query. We have two main HTML files, `input.html`, where users type their query and select ranking operations, and the `output.html`, that displays the document results and their scores for relevance, recency, votes and the final score. Our application has a search bar and clickable links to the original reddit post.

Our application uses PyLucene as backend so that there is a full text-based search on the indexed Reddit data. The flask server essentially handles the user input, extracts the query as well as the ranking parameters to be put into the retrieval function. Then, the top 10 documents are returned and displayed, with a snippet of the post body, subreddit, author and the link to the original post. Our implementation allows us to meet the project requirements for this part, as well as the extra credit, creating an web application that does the information retrieval and shows the results in an easily navigable interface.

VI. Screenshots showing the system in action.

Start/Search Page With Specified Input Settings:

The screenshot shows the 'Reddit Lucene Search' interface. It features a search bar with the placeholder text 'Search Reddit...', a dropdown menu set to 'Combined', and three weight input fields: 'Relevance Wt:' set to '0.5', 'Time Wt:' set to '0.3', and 'Votes Wt:' set to '0.2'. A 'Search' button is located at the bottom center.

Results Page With Post Preview and Link:

The screenshot shows the search results page for '2024 Grammys'. The results are sorted by combined relevance, time, and votes. The top result is from r/grammys with a title '2024 Grammys'. The post preview shows a paragraph about Joni Mitchell's performance and a 'View Post' link. Below the preview, the following scores are listed: BM25 Score: 11.567, Recency Score: 1.000, Votes Score: 3.497, and Final Rank Score: 6.783. The second result is from r/indieheads with a title 'St. Vincent Surprises Fans by Thanking Wife and Daughter at Grammys: "We've Kept It Under Wraps"'. The post preview shows a paragraph about St. Vincent's performance and a 'View Post' link. Below the preview, the following scores are listed: BM25 Score: 11.567, Recency Score: 1.000, Votes Score: 3.497, and Final Rank Score: 6.783.

Search with combined (R wt: 0.5, T wt: 0.3, V wt: 0.2)

The screenshot shows the search results page for 'Kendrick Lamar, Drake & Childish Gambino'. The results are sorted by combined relevance, time, and votes. The top result is from r/hiphopheads with a title 'Kendrick Lamar, Drake & Childish Gambino all declined offers to perform at the 2019 Grammys'. The post preview shows a paragraph about the artists' decline and a 'View Post' link. Below the preview, the following scores are listed: BM25 Score: 6.344, Recency Score: 1.000, Votes Score: 9.588, and Final Rank Score: 9.588. The second result is from r/popheads with a title 'The Weeknd speaks out against the Grammys'. The post preview shows a paragraph about The Weeknd's statement and a 'View Post' link. Below the preview, the following scores are listed: BM25 Score: 7.079, Recency Score: 1.000, Votes Score: 8.146, and Final Rank Score: 8.146.

Search with Upvotes (Votes)

The screenshot shows the search results page for '2024 Grammys'. The results are sorted by votes. The top result is from r/grammys with a title '2024 Grammys'. The post preview shows a paragraph about Joni Mitchell's performance and a 'View Post' link. Below the preview, the following scores are listed: BM25 Score: 11.567, Recency Score: 1.000, Votes Score: 3.497, and Final Rank Score: 1.000. The second result is from r/grammys with a title 'why you should be mad about the grammys'. The post preview shows a paragraph about Jay Z's statement and a 'View Post' link. Below the preview, the following scores are listed: BM25 Score: 11.567, Recency Score: 1.000, Votes Score: 3.497, and Final Rank Score: 1.000.

Search with Recency Score (Time)

The screenshot shows the search results page for '2024 Grammys'. The results are sorted by BM25. The top result is from r/grammys with a title '2024 Grammys'. The post preview shows a paragraph about Joni Mitchell's performance and a 'View Post' link. Below the preview, the following scores are listed: BM25 Score: 11.567, Recency Score: 1.000, Votes Score: 3.497, and Final Rank Score: 11.567. The second result is from r/grammys with a title 'why you should be mad about the grammys'. The post preview shows a paragraph about Jay Z's statement and a 'View Post' link. Below the preview, the following scores are listed: BM25 Score: 11.567, Recency Score: 1.000, Votes Score: 3.497, and Final Rank Score: 11.567.

Search with Relevance (BM25)

The results shown in the screenshot for the query “grammys” are relevant because our query parser found the term in the fields of “Title”, “Body”, and “Comments.” For example, the first result has “Grammys” in the title of the reddit post. In these screenshots, we show you the ranking for “Grammys” with all possible searches: relevancy (BM25 score), recency (timestamps), votes (number of upvotes), and combined ($w_r * \text{bm25} + w_t * \text{recency} + w_v * \text{votes_sc}$). The first result is therefore ranked first because it has “grammys” multiple times in the body as well as in the title, meaning, this outweighs the rest of the scores, making it rank first. This same logic can be applied to the rest of the posts, where each score is taken into account.

VII. Video link:

Demo:

<https://drive.google.com/file/d/111vxyqyCJdtExlhnY8NvS-cs6o7E1ihm/view?usp=sharing>